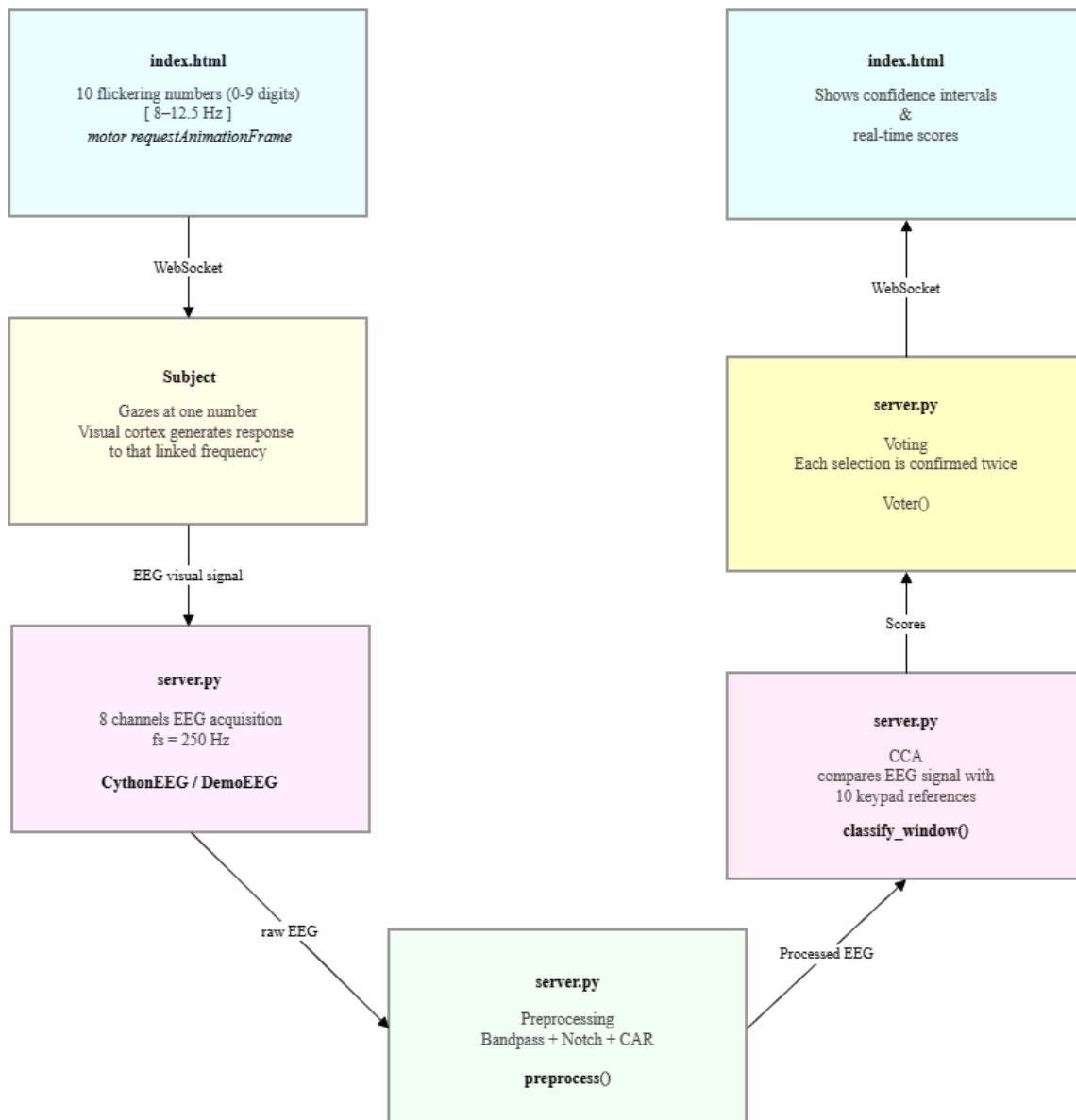


MANUAL SSVEP BCI PHASE 1

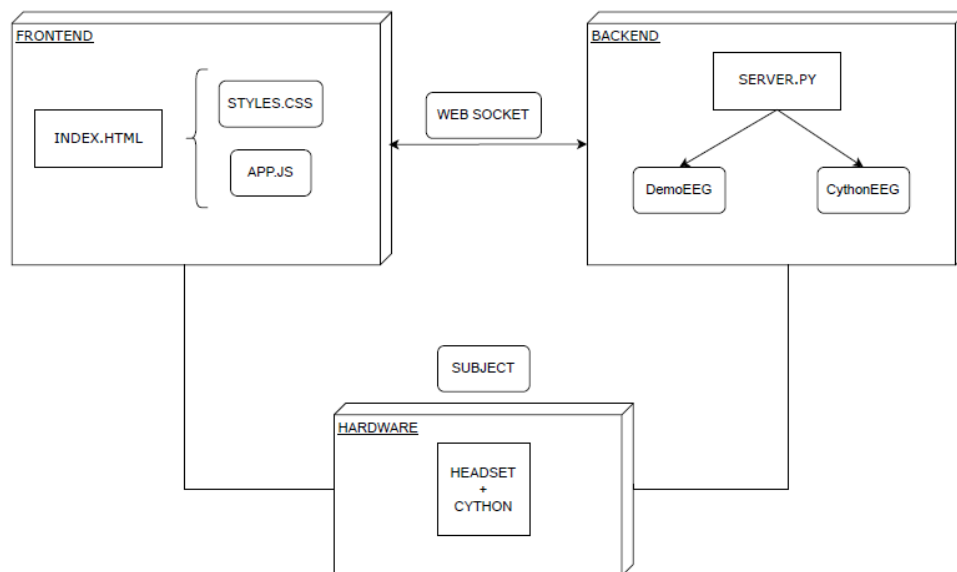
RESUMEN GLOBAL

Workflow completo



Visualización del proyecto

Escritorio > ssvep-bci				
Ordenar Ver ...				
Nombre	Fecha de modificación	Tipo	Tamaño	
index	22/05/2026 2:19	Microsoft Edge HT...	9 KB	
README	22/05/2026 2:11	Archivo de origen ...	2 KB	
server	22/05/2026 2:11	Archivo de origen ...	11 KB	



Visualización del servidor

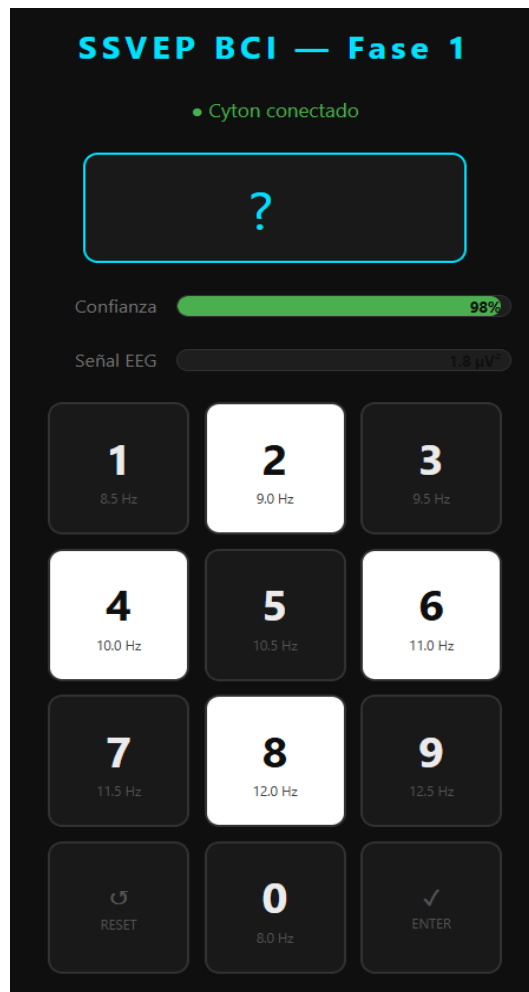
```
Símbolo del sistema - python x + v
Microsoft Windows [Versión 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\carne> cd Desktop\ssvep-bci

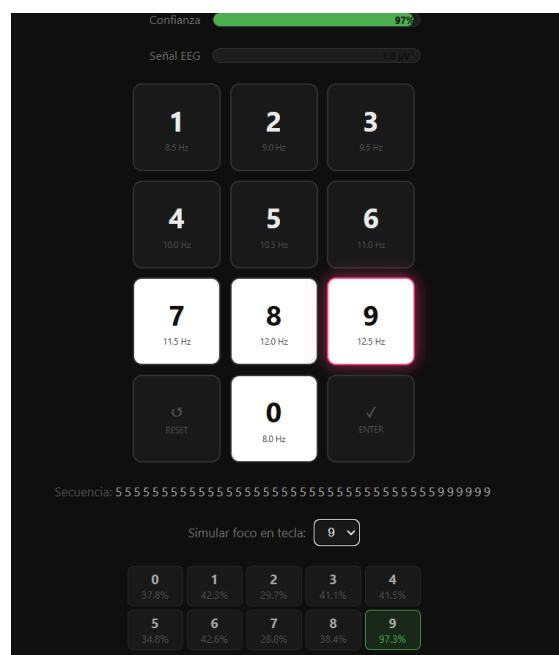
C:\Users\carne\Desktop\ssvep-bci> python server.py
=====
SSVEP BCI - Fase 1 | Modo: DEMO
Ventana: 2s | Armónicos: 3
Umbral: 0.25 | Votos: 2
Abre index.html en el navegador
=====
EEG simulado - usa el selector en la UI para cambiar tecla

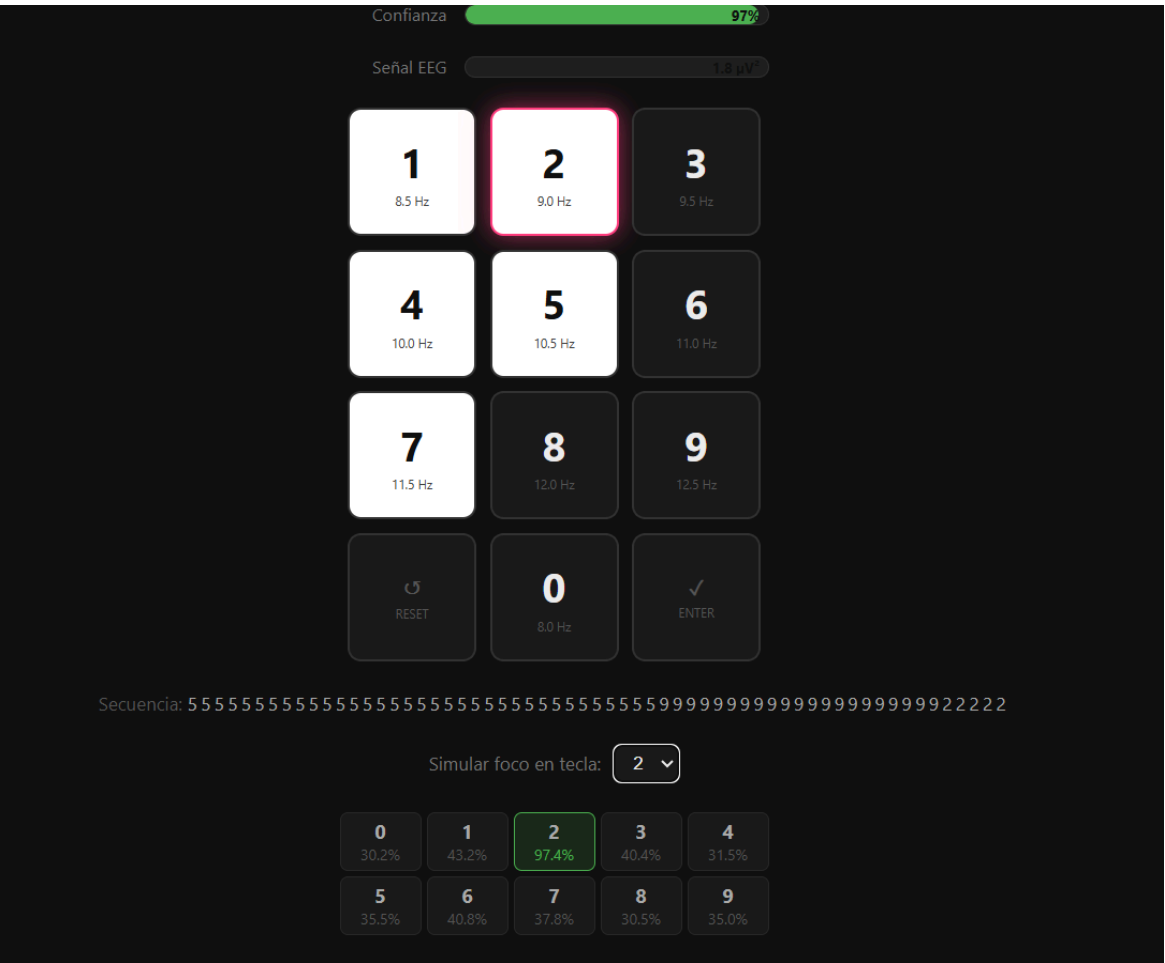
✓ Cliente conectado: ('::1', 61525, 0, 0)
X Cliente desconectado.
|
```

Visualización de la UI web

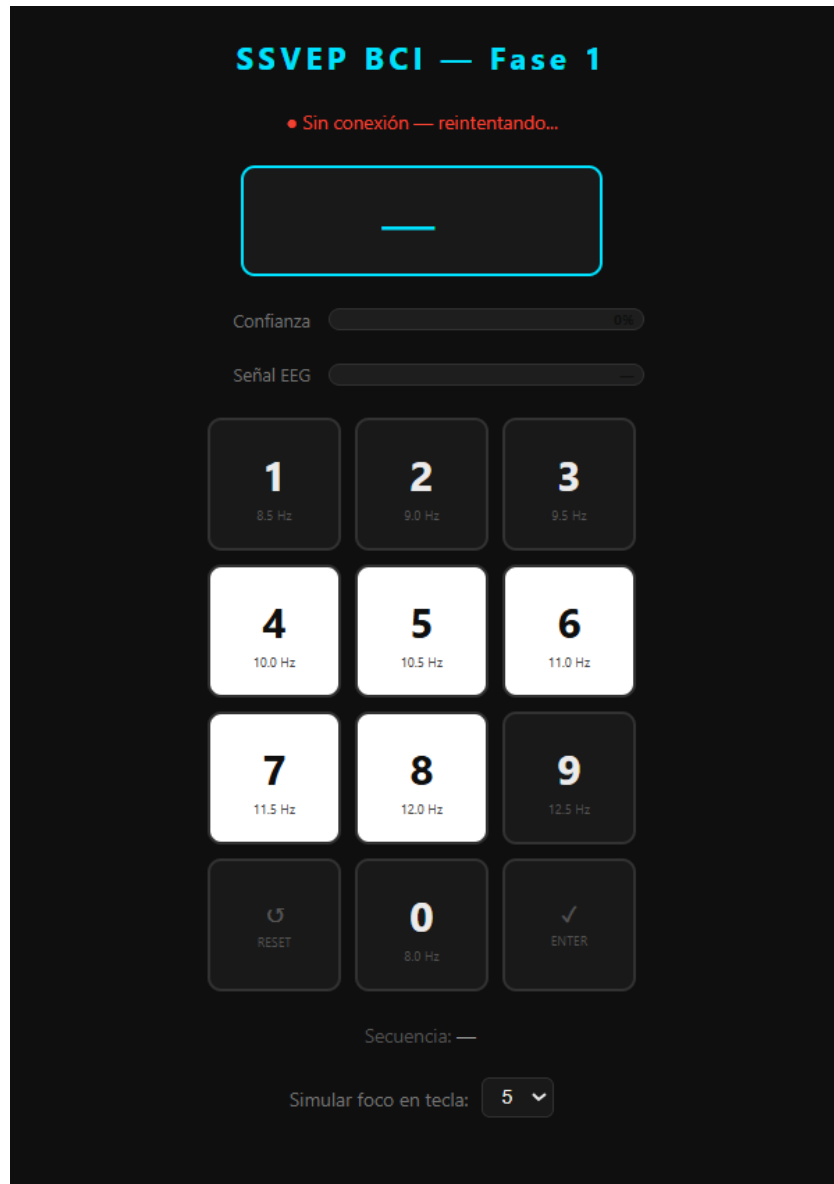


Selección de otra clave numérica



[illegible]

Desconexión con el servidor



SERVER.PY — El cerebro del sistema

BLOQUE 1 — Configuración general (líneas 1–15)

```
MODE = "DEMO"  
SERIAL_PORT = "COM3"
```

Qué hace: actúa como el interruptor principal del sistema.

- `MODE = "DEMO"` → usa EEG inventado por el ordenador (para desarrollar código sin necesidad de usar a la vez hardware).
- `MODE = "HARDWARE"` → usa la placa Cyton real conectada por el USB Dongle al PC.
- `SERIAL_PORT` → el puerto donde está conectada la placa Cyton a través de BLE por USB (COM4 / COM5 dependiendo del portátil empleado).

Gracias a una sola línea de código se permite cambiar entre el proceso de simulación y la conexión con el hardware real (no testada de momento). Esto permite desarrollar el proyecto y la aplicación, validándola y testeándola sin necesitar de depender del casco físico.

BLOQUE 2 — Parámetros de adquisición (líneas 17–30)

```
FS          = 250          # Hz
N_CHANNELS  = 8
WINDOW_SEC  = 2
WINDOW      = FS * WINDOW_SEC  # = 500 muestras
```

Qué hace este bloque de código:

- `FS = 250` → la placa Cyton graba 250 muestras de EEG por segundo.
- `N_CHANNELS = 8` → se usan 8 electrodos disponibles: Fp1, Fp2, C3, C4, P7, P8, O1, O2.
- `WINDOW_SEC = 2` → el clasificador analiza ventanas de tamaño de 2 segundos.
- `WINDOW = 500` → $2 \text{ segundos} \times 250 \text{ muestras/segundo} = 500 \text{ muestras}$ por análisis.

Estos parámetros siguen las características de la placa Cython de la marca OpenBCI: la frecuencia de muestreo es de 250 Hz, se han seleccionado en fases experimentales del TFG ventanas de 2 segundos y por lo tanto se analizarán 500 puntos de datos por cada canal.

BLOQUE 3 — Parámetros de filtrado (líneas 32–37)

```
BP_LOW      = 6.0      # Hz
BP_HIGH     = 40.0     # Hz
NOTCH_FREQ  = 50.0     # Hz
```

Qué hace: Define los límites de los filtros que limpian la señal EEG.

- BP_LOW = 6 → elimina toda señal más lenta que 6 Hz (deriva/movimiento).
- BP_HIGH = 40 → elimina todo lo más rápido que 40 Hz (tensión muscular también llamada EMG).
- NOTCH_FREQ = 50 → elimina exactamente 50 Hz (red de interferencia eléctrica española).

Habitualmente las señales de tipo Electroencefalograma (EEG), llegan sucias o contaminadas. Estos tres parámetros definen los filtros que limpiarán la señal antes de clasificarla.

BLOQUE 4 — Frecuencias de estímulo (líneas 39–44)

```
FREQS = {
    "0": 8.0,  "1": 8.5,  "2": 9.0,  "3": 9.5,  "4": 10.0,
    "5": 10.5, "6": 11.0, "7": 11.5, "8": 12.0, "9": 12.5,
}
```

Qué hace este bloque: Asigna una frecuencia de parpadeo única a cada tecla numérica (0-9).

- El 0 parpadea a 8.0 Hz (8 veces por segundo).
- El 5 parpadea a 10.5 Hz.
- El 9 parpadea a 12.5 Hz.

El rango entre 8–12.5 Hz es la zona donde el cerebro responde mejor a estímulos visuales (SSVEP). Esto se debe a que por debajo de este intervalo interfiere la banda de frecuencias alfa y por encima la respuesta tiende a debilitarse por ruido muscular u otros artefactos.

Se separan las claves/números en 0.5 Hz porque la ventana es de 2 segundos, por lo tanto la resolución matemática es $1/2 = 0.5$ Hz. También se debe a que frecuencias muy cercanas serían indistinguibles por el clasificador de señales EEG.

BLOQUE 5 — Parámetros CCA (líneas 46–60)

```
N_HARMONICS      = 3
N_COMPONENTS      = 1
CONFIDENCE_THRESHOLD = 0.25
N_VOTES           = 2
```

Qué hace:

- `N_HARMONICS = 3` → el cerebro responde a f , $2f$ y $3f$. Es decir, si el usuario mira a un estímulo asociado a 10 Hz, el cerebro también responde a 20 Hz y 30 Hz. Por ello, emplear las frecuencias armónicas mejora la detección.
- `N_COMPONENTS = 1` → CCA extrae 1 componente principal (estándar en literatura SSVEP).
- `CONFIDENCE_THRESHOLD = 0.25` → si la correlación máxima es menor de 0.25, el sistema no emite ninguna predicción. De esta manera, se evitan errores por artefactos.
- `N_VOTES = 2` → el sistema necesita confirmar la misma tecla 2 veces seguidas antes de mostrarla. Esto reduce la aparición de falsos positivos.

Estos parámetros se irán optimizando experimentalmente a lo largo de las fases de desarrollo del sistema BCI hasta que se obtenga la arquitectura más óptima.

BLOQUE 6 — EEG simulado, clase DemoEEG (líneas 63–85)

```
class DemoEEG:
    def __init__(self):
        self.target = "5"

    def set_target(self, key):
```



```

        if key in FREQS:
            self.target = key

    def get_window(self):
        ...
        amp = 1.5 if i >= 6 else 0.4
        sig = (amp * np.sin(...)) + ruido)

```

Qué hace: Genera EEG sintético que imita a un sujeto mirando una tecla concreta.

- `self.target = "5"` → por defecto simula que el sujeto mira el 5.
- `set_target(key)` → cuando se cambia el selector en la UI, esta función actualiza la tecla simulada en la secuencia.
- `amp = 1.5 if i >= 6` → los canales 7 y 8 (O1, O2, occipitales) tienen más amplitud porque en la realidad los electrodos occipitales captan más señal de tipo SSVEP.

El simulador imita lo que haría el EEG real: se muestra una señal ligada a electrodos occipitales con ruido gaussiano añadido.

BLOQUE 7 — Hardware real, clase CytonEEG (líneas 88–108)

```

class CytonEEG:
    def __init__(self):
        from brainflow.board_shim import BoardShim...
        params.serial_port = SERIAL_PORT
        self.board.prepare_session()
        self.board.start_stream()

    def get_window(self):
        data = self.board.get_current_board_data(WINDOW)
        return eeg[:, -WINDOW:]

```

Qué hace este código: Se conecta a la placa Cyton real y lee la señal EEG.

- `prepare_session()` → inicializa la conexión con el hardware.
- `start_stream()` → empieza a grabar las señales de tipo EEG.
- `get_current_board_data(500)` → selecciona las últimas 500 muestras del buffer.

- `eeg[:, -WINDOW:]` → asegura que siempre se devuelve exactamente 500 muestras.

BrainFlow es una herramienta y conjunto de librerías colaboradora con OpenBCI que abstrae el hardware y procesa la señal. Llamando a `get_window()` se reciben los datos en formato numpy.

BLOQUE 8 — Preprocesado (líneas 111–130)

```
def bandpass(x):  
    b, a = butter(4, [6/125, 40/125], btype="band")  
    return filtfilt(b, a, x)  
  
def notch(x):  
    b, a = butter(4, [49/125, 51/125], btype="bandstop")  
    return filtfilt(b, a, x)  
  
def car(eeg):  
    return eeg - eeg.mean(axis=0, keepdims=True)  
  
def preprocess(eeg):  
    filtered = np.array([notch(bandpass(ch)) for ch in eeg])  
    return car(filtered)
```

Qué hace cada función:

`bandpass(x)` → Filtro paso banda

- Deja pasar solo frecuencias entre 6 y 40 Hz.
- Butterworth orden 4 = filtro suave sin distorsión de la fase (gracias a `filtfilt`).
- Se aplica a cada canal.

`notch(x)` → Filtro de eliminación de ruido eléctrico

- Elimina una banda muy estrecha: 49–51 Hz (justo alrededor de 50 Hz)
- Elimina la interferencia de la red eléctrica española

`car(eeg)` → Common Average Reference

- Calcula la media de todos los canales en cada instante.

- La resta de cada canal.
- Resultado: cada canal queda limpio del ruido que comparten todos.

`preprocess(eeg)` → Pipeline completo

- Para cada canal: primero bandpass, luego notch.
- Luego CAR sobre todos los canales juntos.
- El orden importa: primero se filtra y luego se re-referencia.

Estas tres capas son de limpieza de la señal. Primero se eliminan las frecuencias fuera del rango útil, luego la red eléctrica y finalmente el ruido espacial común a todos los electrodos.

BLOQUE 9 — Señal de referencia CCA (líneas 133–143)

```
def ref_signal(freq):
    t = np.arange(WINDOW) / FS
    ref = []
    for n in range(1, N_HARMONICS + 1):
        ref.append(np.sin(2 * np.pi * n * freq * t))
        ref.append(np.cos(2 * np.pi * n * freq * t))
    return np.vstack(ref)
```

Qué hace este código: Construye una señal sintética a modo de comparación con la señal EEG.

- Para cada armónico $n=1,2,3$ genera: seno($2\pi \cdot n \cdot f \cdot t$) y coseno($2\pi \cdot n \cdot f \cdot t$).
- Resultado: 6 señales de referencia (2 por armónico $\times$ 3 armónicos).
- Se utilizan seno y coseno porque la respuesta SSVEP puede tener cualquiera de estas fases (revisar literatura científica que lo explica).

El algoritmo matemático CCA mide cuánto se parece la señal EEG real a la señal de referencia respecto a la frecuencia.

BLOQUE 10 — Clasificador CCA (líneas 145–165)

```
def classify_window(eeg):
    X = eeg.T
    scores = {}
```

```

for label, freq in FREQS.items():
    Y = ref_signal(freq).T
    cca = CCA(n_components=1, max_iter=1000)
    cca.fit(X, Y)
    U, V = cca.transform(X, Y)
    r = float(np.corrcoef(U.T, V.T)[0, 1])
    scores[label] = round(r, 3)

top = max(scores, key=scores.get)
conf = scores[top]
label = top if conf >= CONFIDENCE_THRESHOLD else None
return label, conf, scores

```

Qué hace este clasificador paso a paso:

1. Para cada una de las 10 teclas, se crea su señal de referencia.
2. CCA calcula la correlación máxima entre el EEG y esa referencia (oscila entre 0-1).
3. Guarda los 10 scores.
4. Elige la tecla con el score más alto.
5. Si ese score es menor que 0.25 entonces devuelve None (sin predicción).
6. Si supera 0.25 entonces devuelve la etiqueta ganadora.

Ejemplo: Si miras el 5 (10.5 Hz):

- Score tecla 5: 0.97 ← ganador
- Score tecla 4: 0.31
- Score tecla 6: 0.33
- Resto: ~0.30

CCA realiza 10 comparaciones, una por cada tecla numérica. Gana la que tiene mayor correlación con el EEG real, siempre que supere el umbral mínimo de confianza.

BLOQUE 11 — Clase Voter, votación temporal (líneas 168–182)

```

class Voter:
    def __init__(self):
        self._buf = [] # lista vacía al inicio

    def vote(self, label):
        if label is None:

```

```

        self._buf = []    # si hay artefacto, resetea
        return None

    self._buf.append(label)    # añade la predicción al buffer

    if len(self._buf) < N_VOTES:
        return None          # aún no hay suficientes votos

    winner = max(set(self._buf), key=self._buf.count)
    self._buf = []          # resetea para la siguiente
ronda
    return winner

```

Qué hace este código:

Sabiendo que por ejemplo solo se aceptan 2 votos (N_VOTES = 2):

- Cada 500 ms llega una clasificación (un "voto").
- Si llega None (artefacto detectado) → se vacía la urna y se empieza de cero.
- Si llega una tecla válida → se mete en la urna.
- Cuando hay 2 votos → se cuenta cuál ganó por mayoría y se muestra.
- La urna se vacía para la siguiente selección.

Ejemplo:

- Clasificación 1: "5" → urna: ["5"] → sin resultado todavía
- Clasificación 2: "5" → urna: ["5","5"] → ganador: "5" → se muestra en pantalla

Esto se realiza porque una sola ventana de EEG puede equivocarse por un parpadeo o movimiento involuntario en el sujeto. Si se exigen 2 confirmaciones seguidas se eliminan muchos errores puntuales.

BLOQUE 12 — Servidor WebSocket (líneas 185–215)

```

async def handler(ws, source):
    voter = Voter()
    while True:
        # 1. Escucha mensajes del navegador
        raw = await asyncio.wait_for(ws.recv(), timeout=0.01)
        msg = json.loads(raw)
        if msg.get("type") == "set_target":

```

```
source.set_target(str(msg["key"]))
```

```
# 2. Pipeline completo
raw_eeg = source.get_window()
clean   = preprocess(raw_eeg)
label, conf, scores = classify_window(clean)
voted   = voter.vote(label)
```

```
# 3. Envía resultado al navegador
await ws.send(json.dumps({...}))
await asyncio.sleep(0.5)
```

Qué hace cada paso:

1. El servidor escucha si el usuario cambió la tecla en el selector UI (Demo).
2. Se ejecuta el pipeline: se recoge EEG → se limpia → clasifica → se vota
3. Se envía el resultado al navegador html cada 500 ms.

El servidor se trata de un bucle infinito que cada medio segundo lee la señal EEG, la procesa, la clasifica y manda el resultado a la página web (ver a continuación).

INDEX.HTML — La interfaz visual (UI)

BLOQUE 1 — Estímulos visuales y motor de parpadeo (líneas ~160–185)

```
const flickerKeys =
Array.from(document.querySelectorAll('.key[data-freq]'))
  .filter(el => parseFloat(el.dataset.freq) > 0)
  .map(el => ({
    el,
    period: 1000 / (parseFloat(el.dataset.freq) * 2),
    elapsed: 0,
    state: false,
  }));

function flicker(ts) {
  ...
  k.state = !k.state; // alterna entre blanco y negro
  requestAnimationFrame(flicker);
}
```

Qué hace este bloque de código:

- Lee la frecuencia de cada tecla del HTML (data-freq="10.5").
- Calcula el período: $1000\text{ms} / (10.5 \times 2) = 47.6\text{ms}$ por semiciclo.
- Cada vez que pasa ese tiempo, se alterna la tecla entre blanco y negro.
- `requestAnimationFrame` se sincroniza con la pantalla (60Hz) para conseguir una máxima precisión.

`requestAnimationFrame` está sincronizado con la tasa de refresco del monitor, lo que garantiza que las frecuencias de parpadeo sean más exactas (60 veces por segundo).

BLOQUE 2 — Conexión WebSocket (líneas ~188–205)

```
let socket = null;

function connect() {
  socket = new WebSocket('ws://localhost:8765');

  socket.onopen = () => {
    // muestra "Cyton conectado" en verde
    sendTarget(document.getElementById('demo-target').value);
  };

  socket.onclose = () => {
    // muestra "Sin conexión" en rojo
    socket = null;
    setTimeout(connect, 2000); // reintenta cada 2 segundos
  };

  socket.onmessage = e => updateUI(JSON.parse(e.data));
}
```

Qué hace:

- `socket = new WebSocket(...)` → abre el canal de comunicación con el servidor Python (server.py).
- `onopen` → cuando conecta se muestra verde y se envía la tecla inicial.
- `onclose` → cuando se cae se muestra rojo y se reintenta la conexión automáticamente cada 2s.
- `onmessage` → cuando llega una predicción se actualiza la UI.

Se usa WebSocket como conexión permanente bidireccional entre el navegador HTML y el servidor en Python (se transfieren los datos continuamente).

BLOQUE 3 — Enviar tecla al servidor (líneas ~207–212)

```
function sendTarget(key) {  
  if (socket && socket.readyState === WebSocket.OPEN) {  
    socket.send(JSON.stringify({ type: 'set_target', key: key }));  
  }  
}
```

Qué hace:

- Cuando se cambia el selector en la UI, esta función envía un mensaje JSON al servidor.
- `socket.readyState === WebSocket.OPEN` → comprueba que la conexión esté abierta antes de enviar.
- El servidor recibe el mensaje y por lo tanto cambia la tecla que simula.

El frontend le dice al backend qué tecla se debe simular. Esta comunicación se lleva a cabo en ambas direcciones.

BLOQUE 4 — Actualizar la interfaz (líneas ~215–250)

```
function updateUI(msg) {  
  // 1. Muestra el número detectado  
  document.getElementById('output').textContent = label ?? '?';  
  
  // 2. Barra de confianza  
  bc.style.width = pct + '%';  
  bc.style.background = pct > 60 ? '#4caf50' : pct > 35 ?  
  '#ff9800' : '#f44336';  
  
  // 3. Resalta la tecla predicha en rosa  
  el.classList.add('predicted');  
  
  // 4. Acumula en el buffer de secuencia  
  buffer += label;
```



```
// 5. Muestra scores de todas las frecuencias
renderScores(msg.scores, label);
}
```

Qué hace cada parte:

1. Output → muestra el número ganador en grande (o ? si no hay predicción).
2. Barra de confianza → es verde si es >60%, naranja si está 35–60% o roja si es <35%.
3. Highlight rosa → rodea la tecla detectada.
4. Buffer → acumula los números detectados en la secuencia (ver abajo).
5. Scores → muestra la tabla con el % de correlación de cada frecuencia.

Cada mensaje que llega del servidor actualiza 5 elementos visuales simultáneamente: el display, la barra, la tecla resaltada, la secuencia y la tabla de scores.

BLOQUE 5 — El HTML del teclado (líneas ~110–130)

```
<div class="key" data-key="5" data-freq="10.5">5<small>10.5
Hz</small></div>
```

Qué hace:

- `data-key="5"` → identifica la tecla para el highlight de la predicción.
- `data-freq="10.5"` → le dice al motor de parpadeo a qué frecuencia debe parpadear.
- `<small>10.5 Hz</small>` → muestra la frecuencia visible al usuario.

Cada tecla tiene asociada su frecuencia embebida en el HTML.

INFORMACIÓN ADICIONAL

Revisión de literatura científica en la toma de decisiones

El Paper de **Cheng et al. (2002) [11]** es la inspiración clave del diseño y desarrollo de teclado telefónico SSVEP, a pesar de recurrir a metodologías diferentes. En vez de utilizar CCA recurre a un umbral de amplitud respecto a FFT.

Existen rasgos comunes empleados como los siguientes:

- El rango de frecuencias 6–14 Hz es compatible con el usado en este proyecto que oscila entre 8–12.5 Hz.
- El uso de O1 / O2 como canales principales se contempla en el paper de Cheng y también en este proyecto.
- La idea de comparar entre varias clasificaciones antes de confirmar la selección (en el paper de Cheng se cita "4 consecutive FFTs") es el antecedente conceptual de la variable empleada N_VOTES.
- El tradeoff accuracy / speed son variables necesarias que se contemplan.

Por lo tanto, este estudio proporciona de forma sólida la arquitectura general del sistema (teclado telefónico, uso de O1 y O2, necesidad de confirmar varias clasificaciones, tradeoff velocidad / precisión).

Parámetros concretos justificados por literatura:

N_HARMONICS = 3 y el uso de CCA: la referencia directa es **[12] Chen et al. (2015)**, que es el paper que usa FBCCA donde CCA con armónicos es el método central utilizado. También **[16] Nakanishi et al. (2018)**. Estos dos documentos son referencias del método CCA empleado combinado con armónicos.

WINDOW_SEC = 2 y el tradeoff accuracy / ITR: **[12]** y **[16]** discuten estos temas. Además, el rango 1–4 s está documentado en ambos.

FREQS 8–12.5 Hz, separación de 0.5 Hz: **[11] Cheng et al.** justifica el rango (para incluir el rango de estímulo visual alfa). La separación mínima de 0.5 Hz viene directamente de la resolución espectral $1/\text{WINDOW_SEC}$, que es consecuencia de seleccionar ventanas de 2 s.

CONFIDENCE_THRESHOLD = 0.25: este parámetro es muy dependiente de cada sistema. **[12]** y **[16]** mencionan umbrales pero no dan 0.25 como valor universal. Por ello, se asume este valor inicial experimentalmente en el proyecto

y se optimizará, lo cual está respaldado por [11] ("it is better to optimize the detection criteria for each user individually").

N_VOTES = 2: el concepto viene de [11] (se menciona "consecutive FFTs") pero la votación por mayoría como mecanismo explícito se ha escogido como método robusto. Se trata de una decisión de diseño que se debe validar experimentalmente.

BP 6–40 Hz, notch 50 Hz: [11] usa 4–35 Hz. El rango escogido en el proyecto de 6–40 Hz se puede considerar estándar; para el notch a 50 Hz cualquier paper europeo lo menciona. [16] **Nakanishi** reporta filtros similares.

Referencias

Estas referencias son extraídas de la documentación académica oficial del TFG que se está desarrollando paralelamente.

- [1] Nicolas-Alonso LF, Gomez-Gil J. Brain Computer Interfaces, a Review. *Sensors*. 2012;12(2):1211–1279.
- [2] Saha S, Mamun KA, Ahmed K, Mostafa R, Naik GR, Darvishi S, Khandoker AH, Baumert M. Progress in Brain Computer Interface: Challenges and Opportunities. *Front Syst Neurosci*. 2021;15:578875.
- [3] Vărbu K, Muhammad N, Muhammad Y. Past, Present, and Future of EEG-Based BCI Applications. *Sensors*. 2022;22(9):3331.
- [4] Mridha MF, Das SC, Kabir MM, Lima AA, Islam MR, Watanobe Y. Brain-Computer Interface: Advancement and Challenges. *Sensors*. 2021;21(17):5746.
- [5] Chen S, Chen M, Wang X, Liu X, Liu B, Ming D. Brain-computer interfaces in 2023–2024. *[journal pending verification]*. 2024.
- [6] Rashid U, Niazi IK, Signal N, Taylor D. An EEG Experimental Study Evaluating the Performance of Texas Instruments ADS1299. *Sensors*. 2018;18(11):3721.
- [7] Schalk G, McFarland DJ, Hinterberger T, Birbaumer N, Wolpaw JR. BCI2000: A General-Purpose Brain-Computer Interface (BCI) System. *IEEE Trans Biomed Eng*. 2004;51(6):1034–1043.

- [8] Memmott T, Koçanaoğulları A, Lawhead M, Klee D, Dudy S, Fried-Oken M, Oken B. BciPy: brain-computer interface software in Python. *Brain-Computer Interfaces*. 2021;8(4):137–153.
- [9] Cruz MV, Jamal S, Sethuraman SC. A Comprehensive Survey of Brain-Computer Interface Technology in Health care. *[journal pending verification]*. 2023.
- [10] Sosa Rojo MV. Evaluación de electrodos EEG secos, húmedos y con gel para la implementación de sistemas BCI basados en potenciales evocados P300. *TFG, Grado en Ingeniería Biomédica, Universidad de Valladolid*. 2024.
- [11] Cheng M, Gao X, Gao S, Xu D. Design and Implementation of a Brain-Computer Interface With High Transfer Rates. *IEEE Trans Biomed Eng*. 2002;49(10):1181–1186.
- [12] Chen X, Wang Y, Gao S, Jung TP, Gao X. Filter bank canonical correlation analysis for implementing a high-speed SSVEP-based brain-computer interface. *J Neural Eng*. 2015;12(4):046008.
- [13] Islam MR, Molla MKI, Nakanishi M, Tanaka T. Unsupervised frequency-recognition method of SSVEPs using a filter bank implementation of binary subband CCA. *J Neural Eng*. 2017;14(2):026007.
- [14] Jalilpour S, Hajipour Sardouie S, Mijani A. A novel hybrid BCI speller based on RSVP and SSVEP paradigm. *Comput Methods Programs Biomed*. 2020;187:105326.
- [15] Fodor MA, Cantürk A, Heisenberg G, Volosyak I. Streamlining cVEP Paradigms: Effects of a Minimized Electrode Montage on Brain-Computer Interface Performance. *Sensors*. 2023;23(4):2256.
- [16] Nakanishi M, Wang Y, Chen X, Wang YT, Gao X, Jung TP. Enhancing detection of SSVEPs for a high-speed brain speller using task-related component analysis. *IEEE Trans Biomed Eng*. 2018;65(1):104–112.

Design and Implementation of a Brain-Computer Interface With High Transfer Rates

Ming Cheng*, Xiaorong Gao, Shangkai Gao, *Senior Member, IEEE*, and Dingfeng Xu

Abstract—This paper presents a brain-computer interface (BCI) that can help users to input phone numbers. The system is based on the steady-state visual evoked potential (SSVEP). Twelve buttons illuminated at different rates were displayed on a computer monitor. The buttons constituted a virtual telephone keypad, representing the ten digits 0–9, BACKSPACE, and ENTER. Users could input phone number by gazing at these buttons. The frequency-coded SSVEP was used to judge which button the user desired. Eight of the thirteen subjects succeeded in ringing the mobile phone using the system. The average transfer rate over all subjects was 27.15 bits/min. The attractive features of the system are noninvasive signal recording, little training required for use, and high information transfer rate. Approaches to improve the performance of the system are discussed.

Index Terms—Brain-computer interface (BCI), electroencephalography (EEG), steady-state visual evoked potential (SSVEP), transfer rate.

EEG patterns associated with different mental tasks [6], [7], P300 potentials [8], [9], and visual evoked potentials (VEPs) [10], [11]. Electrodes can be placed either on the scalp or on the cortex [10], [12], [13]. Typical BCI applications involve cursor movement, letter or icon selection, or device control. Currently, BCIs are mainly used as an augmentative communication technology for individuals with motor impairments, such as amyotrophic lateral sclerosis (ALS) or cerebral palsy.

Several issues are crucial to further development and expanded utilization of the BCI technology. The first issue is the information transfer rate. Current BCIs are relatively low bandwidth devices, offering maximum information transfer rates of 5–25 bits/min at best [1]. At this rate, it may take several minutes to input a simple word to a computer. If this rate could be increased, BCIs might offer all individuals useful ways to interact with their environment.

The second issue is the training time for users to develop com-

1182

IEEE TRANSACTIONS ON BIOMEDICAL ENGINEERING, VOL. 49, NO. 10, OCTOBER 2002

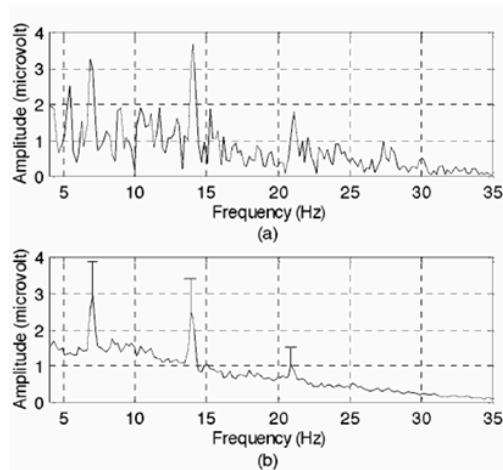


Fig. 1. Amplitude spectrum of SSVEP in response to 7 Hz stimulation. Three peaks at 7 Hz, 14 Hz, and 21 Hz can be found clearly. (a) Single trial amplitude spectrum. Sampling rate is 200 Hz and the trial has 512 data points (zero-padded to 1024 points before FFT). (b) Mean amplitude spectrum averaged over 40 trials. Vertical lines give standard deviation.

buttons modulated at 17.56 and 23.42 Hz were used to induce multiple SSVEPs.

We have applied SSVEP-based BCI to control cursor movements [18]. In that work, four buttons illuminated at different frequencies represented four directions; users could move the cursor to different directions simply by looking at the corresponding buttons. In this paper, we will introduce a new application of SSVEP-based BCI to show the system's ability to provide high transfer rates.

II. METHODS

A. Scientific Background

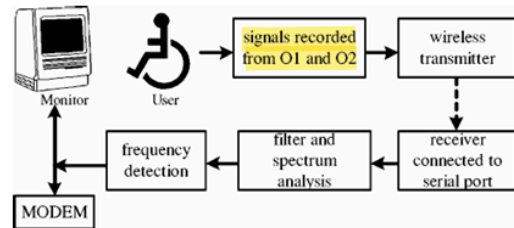


Fig. 2. Block diagram of the SSVEP-based BCI system for phone call.

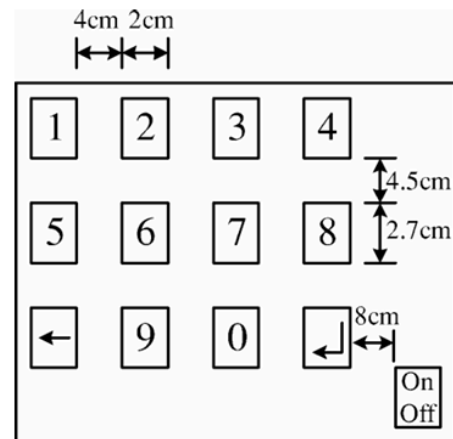


Fig. 3. Thirteen buttons widely spaced on the screen of a computer monitor. The 3×4 stimuli matrix constituted a virtual telephone keypad. The button ON/OFF was designed to control the start and stop of the other stimuli.

stimulus at an attended versus an unattended location [20]. The increased SSVEP amplitudes reflect an enhancement of neural responses to a stimulus that falls within the range of spatial attention. This observation shows that SSVEP may provide an

Currently, many BCIs are designed to control one-dimensional or two-dimensional cursor movements. Only two or four EEG patterns are needed for this kind of task. It is relatively simple to increase the number of selections in VEP-based BCI systems. The system introduced in [10] could provide up to 64 visual stimuli modulated with m-sequences. One trained subject with implanted electrodes reached communication rates of 10–12 words/min. In our system, each trial had 12 possible selections, which was crucial to the realization of high transfer rates. The results of task three suggest that it is possible to place more visual stimuli in a small area so that the transfer rate can be further improved. High stimulation frequencies can be realized using LEDs or CRT monitor with high refresh rate.

In any real time system, there is a tradeoff between accuracy and speed. In our system, a selection was made only if the same repetition frequency was detected in four consecutive FFTs. To increase the speed, it is better to optimize the detection criteria for each user individually. For subjects with strong SSVEP, a higher threshold can be used, and a selection can be made based on only two or three consecutive FFTs. Furthermore, advanced signal processing methods should be used; e.g., the prewhitening of the background brain activity is expected to improve the SNR of SSVEP [21], [22].

Filter bank canonical correlation analysis for implementing a high-speed SSVEP-based brain–computer interface

Xiaogang Chen¹, Yijun Wang^{2,3}, Shangkai Gao¹, Tzyy-Ping Jung² and Xiaorong Gao¹

¹ Department of Biomedical Engineering, School of Medicine, Tsinghua University, Beijing, 100084, People's Republic of China

² Swartz Center for Computational Neuroscience, Institute for Neural Computation, University of California, San Diego, La Jolla, CA 92093, USA

³ State Key Laboratory on Integrated Optoelectronics, Institute of Semiconductors, Chinese Academy of Sciences, Beijing, 100083, People's Republic of China

E-mail: yijun@scn.ucsd.edu and gxr-dea@tsinghua.edu.cn

Received 14 December 2014, revised 13 March 2015

Accepted for publication 29 April 2015

Published 2 June 2015



Abstract

Objective. Recently, canonical correlation analysis (CCA) has been widely used in steady-state visual evoked potential (SSVEP)-based brain–computer interfaces (BCIs) due to its high efficiency, robustness, and simple implementation. However, a method with which to make use of harmonic SSVEP components to enhance the CCA-based frequency detection has not been well established. **Approach.** This study proposed a filter bank canonical correlation analysis (FBCCA) method to incorporate fundamental and harmonic frequency components to improve the detection of SSVEPs. A 40-target BCI speller based on frequency coding (frequency range: 8–15.8 Hz, frequency interval: 0.2 Hz) was used for performance evaluation. To optimize the filter bank design, three methods (M₁: sub-bands with equally spaced bandwidths; M₂: sub-bands corresponding to individual harmonic frequency bands; M₃: sub-bands covering multiple harmonic frequency bands) were proposed for comparison. Classification accuracy and information transfer rate (ITR) of the three FBCCA methods and the standard CCA method were estimated using an offline dataset from 12 subjects. Furthermore, an online BCI speller adopting the optimal FBCCA method was tested with a group of 10 subjects. **Main results.** The FBCCA methods significantly outperformed the standard CCA method. The method M₃ achieved the highest classification performance. At a spelling rate of ~33.3 characters/min, the online BCI speller obtained an average ITR of 151.18 ± 20.34 bits min⁻¹. **Significance.** By incorporating the fundamental and harmonic SSVEP components in target identification, the proposed FBCCA method significantly improves the performance of the SSVEP-based BCI, and thereby facilitates its practical applications such as high-speed spelling.

Enhancing Detection of SSVEPs for a High-Speed Brain Speller Using Task-Related Component Analysis

Masaki Nakanishi, *Member, IEEE*, Yijun Wang*, *Member, IEEE*, Xiaogang Chen, *Member, IEEE*, Yu-Te Wang, *Member, IEEE*, Xiaorong Gao, *Member, IEEE*, and Tzzy-Ping Jung, *Fellow, IEEE*

Abstract— Objective: This study proposes and evaluates a novel data-driven spatial filtering approach for enhancing steady-state visual evoked potentials (SSVEPs) detection towards a high-speed brain-computer interface (BCI) speller. **Methods:** Task-related component analysis (TRCA), which can enhance reproducibility of SSVEPs across multiple trials, was employed to improve signal-to-noise ratio (SNR) of SSVEP signals by removing background electroencephalographic (EEG) activities. An ensemble method was further developed to integrate TRCA filters corresponding to multiple stimulation frequencies. This study conducted a comparison of BCI performance between the proposed TRCA-based method and an extended canonical correlation analysis (CCA)-based method using a 40-class SSVEP dataset recorded from 12 subjects. An online BCI speller was further implemented using a cue-guided target selection task with 20 subjects and a free-spelling task with 10 of the subjects. **Results:** The offline comparison results indicate that the proposed TRCA-based approach can significantly improve the classification accuracy compared with the extended CCA-based method. Furthermore, the online BCI speller achieved averaged information transfer rates (ITRs) of 325.33 ± 38.17 bits/min with the cue-guided task and 198.67 ± 50.48 bits/min with the free-spelling task. **Conclusion:** This study validated the efficiency of the proposed TRCA-based method in implementing a high-speed SSVEP-based BCI. **Significance:** The high-speed SSVEP-based BCIs using the TRCA method have great potential for various applications in communication and control.

Index Terms—Brain-computer interfaces (BCI), electroencephalography (EEG), steady-state visual evoked potentials (SSVEP), task-related component analysis (TRCA).

I. INTRODUCTION

Brain-computer interfaces (BCIs) based on steady-state visual evoked potentials (SSVEPs) have been attracting much attention due to high communication rates [1]–[3]. In a recent study, a 40-target SSVEP-based BCI achieved an information transfer rate (ITR) of 267 bits/min, to our knowledge, the highest ITR reported in BCI research to date [2]. To improve performance of SSVEP-based BCIs, researchers are working on two main directions: (1) increasing the number of classes (e.g., visual stimuli), (2) improving target identification algorithms [4]. Although the number of frequencies that can be presented on a computer monitor is limited by its refresh rate, various stimulus presentation approaches inspired by multiple access (MA) technologies used in telecommunication have succeeded in rendering a large number of visual flickers on a computer monitor [2], [5]–[8]. For example, the frequency shift keying (FSK) method has been applied to increase the number of visual stimuli with different properties using the limited number of available frequencies [5]. A hybrid paradigm which combines SSVEP with other electroencephalogram (EEG) features (e.g., P300) is another way to increase the number of selections [6]. In several other studies, frequency-approximation approaches make it possible to render a large number of flickers with flexible frequencies on a computer monitor [7]–[9]. More recently, the efficacy of a joint frequency-phase modulation

>> HIGH SPEED BCI							
8.0 0.00	9.0 1.75	10.0 1.50	11.0 1.25	12.0 1.00	13.0 0.75	14.0 0.50	15.0 0.25
8.2 0.35	9.2 0.10	10.2 1.85	11.2 1.60	12.2 1.35	13.2 1.10	14.2 0.85	15.2 0.60
8.4 0.70	9.4 0.45	10.4 0.20	11.4 1.95	12.4 1.70	13.4 1.45	14.4 1.20	15.4 0.95
8.6 1.05	9.6 0.80	10.6 0.55	11.6 0.30	12.6 0.05	13.6 1.80	14.6 1.55	15.6 1.30
8.8 1.40	9.8 1.15	10.8 0.90	11.8 0.65	12.8 0.40	13.8 0.15	14.8 1.90	15.8 1.65

Freq.
(Hz)

Phase
(π)

Fig. 1. Stimulus design of the 40-target BCI system. Frequency and phase values specified for each target.

2) Stimulus Design

This study used the sampled sinusoidal stimulation method [9] to present visual flickers coded by the JFPM method on a 23.6-inch liquid-crystal display (LCD) monitor. The resolution and refresh rate of the monitor were 1,920 x 1,080 pixels and 60 Hz, respectively. The user interface is a 5 x 8 matrix of visual stimuli containing 40 characters (26 English alphabet letters, 10 digits, and 4 other symbols). Specifically, 40 stimuli are tagged by linearly increasing frequencies and phases, of which the increments are both proportional to stimulus index (Fig. 1). The frequency range was selected from 8 Hz to 15.8 Hz with an interval of 0.2 Hz. The phase values started from 0 and the phase interval was 0.35π . Each stimulus was rendered within a 140- $\times$ 140-pixel square. The characters were presented within a 32- $\times$ 32-pixel square at the center of each stimulus. The vertical and horizontal distances between two neighboring stimuli were 50 pixels. The stimulus program was developed under MATLAB (MathWorks, Inc.) using the Psychophysics Toolbox Version 3 [29].

3) EEG Recording

EEG data were recorded by nine electrodes over parietal and occipital areas (Pz, PO5, PO3, POz, PO4, PO6, O1, Oz,

The present design of visual stimulator uses stimuli flickering in the alpha band (e.g., 8 - 13 Hz) to achieve high performance. However, visual flickers at higher frequencies above the flicker fusion threshold might be a better choice to reduce visual fatigue and discomfort for practical applications. In general, the amplitude of SSVEPs decreases as stimulus frequency increases [12]. The proposed methods might have great potential to improve the detection accuracy of the high-frequency SSVEPs. In addition, the TRCA-based spatial filtering method could be applied to enhance SNRs of other time-locked EEG components such as event-related potentials (ERPs), which have been widely used in the fields of BCI, cognitive neuroscience, and clinical applications. In particular, P300-based BCIs generally require an averaging process across multiple trials to increase detection accuracy, leading to slow communication speed and low ITR [38]. The number of trials required in the averaging process could be reduced by employing the TRCA-based spatial filter.

V. CONCLUSIONS

A novel SSVEP detection method using the TRCA-based spatial filters was proposed and evaluated in this paper. The offline analysis results showed that the proposed method significantly outperformed the extended CCA-based method in terms of classification accuracy and ITR. The feasibility of the proposed method in online applications was also validated